```
// Run with:  node test/checkout.test.mjs
// Uses a pretend Stripe, so nothing is charged. Every line should say PASS.
import fs from 'node:fs'; import os from 'node:os'; import path from 'node:path';
const here = path.dirname(fileURLToPath(import.meta.url));
const tmp = path.join(os.tmpdir(), 'checkout-under-test.mjs');
fs.copyFileSync(path.join(here, '../functions/api/checkout.js'), tmp);
const { onRequest } = await import(pathToFileURL(tmp).href);

let sent = null, mode = 'ok';
globalThis.fetch = async (url, init) => {
  sent = { url, init, form: new URLSearchParams(init.body) };
  if (mode === 'down') throw new Error('network down');
  if (mode === 'reject') return new Response(JSON.stringify({ error: { message: '
  return new Response(JSON.stringify({ url: 'https://checkout.stripe.com/c/pay/TE
};
const env = { STRIPE_SECRET_KEY: 'sk_test_x', SITE_URL: 'https://example-shop.com
const get = (qs, e = env) => onRequest({ request: new Request('https://shop.test/
const cart = (rows) => JSON.stringify(rows);
const amounts = () => [...sent.form.entries()].filter(([k]) => k.endsWith('[unit_
let failed = 0;
const check = (name, ok, extra = '') => { console.log((ok ? 'PASS ' : 'FAIL ') +

let r = await get({ cart: cart([['Handcrafted Winter Angel', 'Red', 2], ['Large M
check('normal cart redirects to Stripe', r.status === 303 && r.headers.get('Locat
check('prices and shipping are right (8499, 13499, 750)', JSON.stringify(amounts(
r = await get({ cart: cart([['High Heel Ornament', 'Royal Blue', 1]]), zip: '9021
check('High Heel pair is 44.99 (4499)', amounts()[0] === 4499, JSON.stringify(amo
r = await get({ cart: cart([['Handcrafted Winter Angel', 'Red', 2], ['Large Medal
check('request goes to Stripe with the secret key', sent.url === 'https://api.str
check('shipping address + phone collected, US only', sent.form.get('shipping_addr
check('success/cancel links use SITE_URL', sent.form.get('success_url') === 'http
r = await get({ cart: cart([['Handcrafted Winter Angel', 'Red', 1, 0.01]]), zip:
check('a price sent by the browser is ignored', amounts()[0] === 8499, String(amo
for (const [label, c] of [['unknown product', [['Fake Item', 'Red', 1]]], ['negat
  r = await get({ cart: cart(c) }); check(label + ' -> 400', r.status === 400);
}
r = await get({ cart: 'not json' }); check('bad JSON -> 400', r.status === 400);
r = await get({}); check('no cart -> 400', r.status === 400);
r = await onRequest({ request: new Request('https://shop.test/api/checkout', { me
r = await onRequest({ request: new Request('https://shop.test/api/checkout', { me
check('POST returns the Stripe link as JSON', r.status === 200 && (await r.json()
r = await onRequest({ request: new Request('https://shop.test/api/checkout', { me
r = await get({ cart: cart([['Angel Ornament', 'Red', 1]]) }, { SITE_URL: 'https:
```

```
r = await get({ cart: cart([['Angel Ornament', 'Red', 1]]) }, { STRIPE_SECRET_KEY
r = await get({ cart: cart([['Angel Ornament', 'Red', 1]]) }, { STRIPE_SECRET_KEY
mode = 'reject'; r = await get({ cart: cart([['Angel Ornament', 'Red', 1]]) }); c
mode = 'down'; r = await get({ cart: cart([['Angel Ornament', 'Red', 1]]) }); che
mode = 'ok';
await get({ cart: cart([['Angel Ornament', 'Red', 1], ['Guitar Ornament', 'Red',
check('shipping for 2 lines, ZIP 9xxxx = 750 cents', amounts().at(-1) === 750, St

// ---- Gift Box ----
const box = (items, extra = []) => cart([['Ornament Gift Box', 'Red', 1, items],
const nameOf = (i = 0) => sent.form.get(`line_items[${i}][price_data][product_dat
r = await get({ cart: box([['Angel Ornament', 'Red', 1], ['Sapphire Cameo Ornamen
check('box with 3 ornaments = sum of ornaments (25497)', r.status === 303 && amou
check('3+ ornaments adds a FREE separate gift box note', /FREE separate gift box/
r = await get({ cart: box([['Angel Ornament', 'Red', 2]]), zip: '90210' });
check('box with 2 ornaments = 16998 and no free box', amounts()[0] === 16998 && !
r = await get({ cart: box([['Angel Ornament', 'Red', 1], ['Throne Ornament', 'Cre
check('3 ornaments counted by quantity (free box)', /FREE/.test(nameOf()) && amou
r = await get({ cart: cart([['Ornament Gift Box', 'Red', 1]]) }); check('box with
r = await get({ cart: box([]) }); check('box with empty list -> 400', r.status ==
r = await get({ cart: box([['Ornament Gift Box', 'Red', 1]]) }); check('box insid
r = await get({ cart: box([['Fake Ornament', 'Red', 1]]) }); check('unknown ornam
r = await get({ cart: box([['Angel Ornament', 'Red', 13]]) }); check('more than 1
r = await get({ cart: box([['Angel Ornament', 'Red', 1.5]]) }); check('fractional
r = await get({ cart: box([['constructor', 'Red', 1]]) }); check('box item named
r = await get({ cart: cart([['Ornament Gift Box', 'Red', 1, [['Angel Ornament', '
console.log(failed ? '\n' + failed + ' check(s) FAILED' : '\nAll checks passed');
process.exit(failed ? 1 : 0);
```